데이터분석을위한 PANDAS

ABOUT PANDAS

PANDAS 살펴보기

◆ 소개

- 계량 경제학 용어인 '**PAN**el **DA**ta' 앞 글자
- **금융 데이터에 대한 계량적 분석 위해 2008년부터 개발 작업 시작**
- **BSD 라이선스 : 소스코드 공개 의무 없음. 상업적 사용 가능**
- https://pandas.pydata.org

PANDAS 살펴보기

◆ 설치

- **CONDA 기반**

  conda install **-c** **conda-forge** **pandas**

  conda install **pandas**

- **CONDA 저장소**

  conda-forge : 커뮤니티로 최신 버전 많음

  default : 안전성 중심. 최신 버전 없음

PANDAS 살펴보기

◆ 설치

- **가상환경 생성 – Anaconda S/W**

  (1) Anaconda Powrshell 관리자모드로 실행

  (2) 현재 가상환경 리스트

```
(base) PS C:\Users\kwon> conda env list

# conda environments:
#
                         C:\ProgramData\anaconda3
base                   * C:\Users\kwon\anaconda3
PY311                    C:\Users\kwon\anaconda3\envs\PY311
```

PANDAS 살펴보기

◆ 설치

- **가상환경 생성 – Anaconda S/W**

(3) Anaconda Powrshell 관리자모드로 실행

```
(base) PS C:\Users\kwon>  conda  create  -n  DATA_311  python=3.11
```

(4) 가상환경 리스트에서 생성된 가상환경 확인

```
(base) PS C:\Users\kwon> conda env list

# conda environments:
#
                         C:\ProgramData\anaconda3
base                   * C:\Users\kwon\anaconda3
DATA_311                 C:\Users\kwon\anaconda3\envs\DATA_311
PY311                    C:\Users\kwon\anaconda3\envs\PY311
```

PANDAS 살펴보기

◆ 설치

- **가상환경 생성 – Anaconda S/W**

  (5) 새로운 가상환경 실행

```
(base) PS C:\Users\kwon> conda activate DATY_311
```

↓

```
(DATA_311) PS C:\Users\kwon>
```

PANDAS 살펴보기

◆ 설치

- **패키지 설치 – Anaconda S/W**

(6) pandas 설치

```
(DATA_311) PS C:\Users\kwon> conda install pandas
```

(7) 설치 확인 – python 모드에서 체크

```
(DATA_311) PS C:\Users\kwon> python
Python 3.11.14 | packaged by conda-forge | (main, Oct 13 2025, 14:00:26)
[MSC v.1944 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> import pandas
>>> exit( )
(DATA_311) PS C:\Users\kwon>
```

python 명령어 모드

PANDAS 살펴보기

◆ 설치

- **패키지 삭제 후 다시 설치 – Anaconda S/W**

(1) pandas 삭제

```
(DATA_311) PS C:\Users\kwon> conda  uninstall  pandas
## Package Plan ##
  environment location: C:\Users\kwon\anaconda3\envs\DATA_311
  removed specs:
    - pandas
```

(2) 다시 설치

```
(DATA_311) PS C:\Users\kwon> conda install pandas
```

PANDAS 살펴보기

◆ 의존성 패키지들

- **Pandas 동작에 연관된 패키지들**
- **패키지별 가상환경 필요한 이유**

  → 다른 패키지들과 중복되는 경우 버전 충돌 발생 원인
- **반드시 설치**되어야 함

  → NumPy : 다차원 배열 처리 패키지

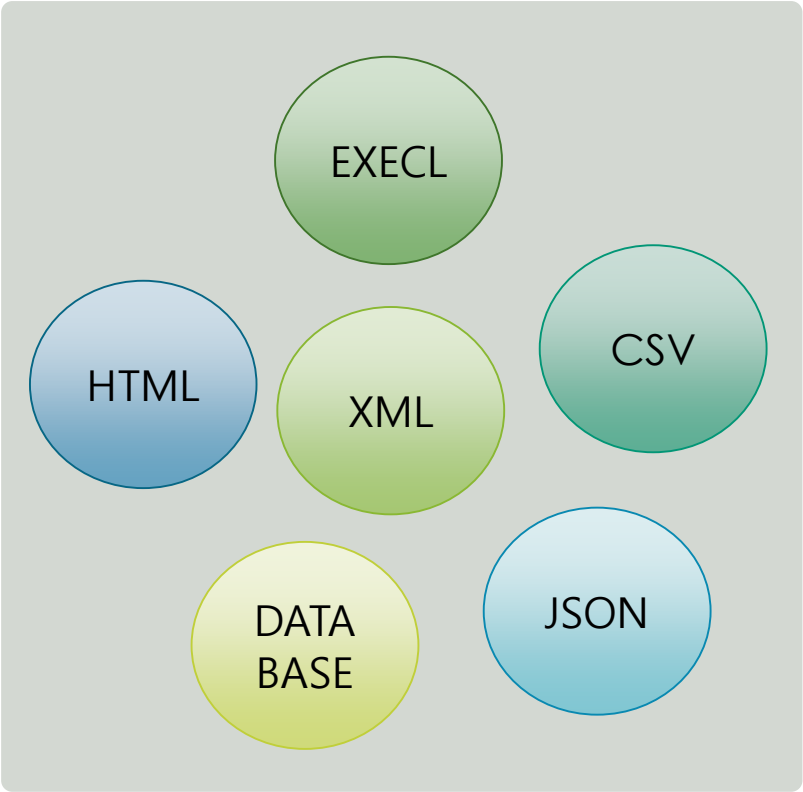
  → python-dateutil : date관련 패키지

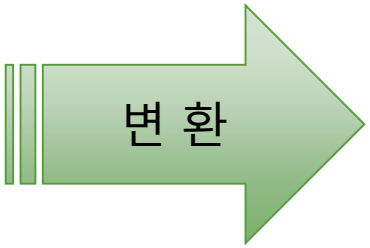
  → pytz : 세계 시간 Timezone 패키지

PANDAS 살펴보기

◆ Pandas 자료구조

[다양한 형태 데이터]





동일형태
동일포맷

번호

번호	이름	학교

PANDAS 살펴보기

◆ 자료구조

- **시리즈 (Series)**

  데이터를 1차원 배열에 저장하는 구조화 데이터

9	홍길동	대구

번호
9
11
21

- **데이터프레임 (DataFrame)**

  데이터를 2차원 배열에 저장하는 구조화 데이터

번호	이름	학교
9	홍길동	대구
11	마징가	부산
21	베트맨	부산

PANDAS 살펴보기

◆ 자료구조

Series 타입

DataFrame 타입

번호	이름	학교
1	홍길동	대구중
9	마징가	부산중
21	원더우먼	마산중

Element
원래 데이터 타입
int, float, bool, str ...

PANDAS 살펴보기

◆ Pandas 데이터 타입 - dType

Pandas	Python	NumPy	사용
object	str or mixed	string, unicode, mixed type	Text or mixed numeric non-numeric values
int64	int	int, int8, int16, int32, int64, uint8, uint16, uint32, uint64	Integer numbers
float64	float	float, float16, float32, float64	Floating point numbers
bool	bool	bool	True/False values
datetime64	NA	datetime64[ns]	Date and time values
timedelta[ns]	NA	NA	Differences between two datetimes
category	NA	NA	Finitelist of text values

PANDAS 데이터 파일 처리

PANDAS 데이터 파일 처리

◆ 파일 입력 & 출력 함수

- 다양한 형태의 외부 파일 읽어 DataFrame 변환
- DataFrame함수로 다양하게 데이터 분석 및 가공
- 파일 형식 : csv, json, xsx 등등

PANDAS 데이터 파일 처리

◆ 파일 입력 & 출력 함수

FILE TYPE	READER	WRITER
CSV	read_csv( )	to_csv( )
JSON	read_json( )	to_json( )
HTML	read_html( )	to_html( )
Local clipboard	read_clipboard( )	to_clipboard( )
Ms Excel	read_excel( )	to_excel( )
HDF5 Format	read_hdf( )	to_hdf( )
SQL	read_sql( )	to_sql( )

PANDAS 데이터 파일 처리

◆ CSV 파일

- 데이터 값을 쉼표(,)로 구분하는 파일
- Comma Separated Values 약자
- **쉼표(,)로 열 구분, 줄바꿈으로 행(row) 구분**
- TSV(Tab), SSV(Space) 데이터 파일도 존재

PANDAS 데이터 파일 처리

◆ CSV 파일

형태 : 데이터, 데이터, 데이터

```
1,13.2,9.1,blue
1,98,2.8,gray
2,10.5,81.3,red
1,8.8,5.21,yellow
```

```
1;13.2;9.1;blue
1;98;2.8;gray
2;10.5;81.3;red
1;8.8;5.21;yellow
```

PANDAS 데이터 파일 처리

◆ CSV 파일

dataframe = Pandas.read_csv("파일 경로(이름)", 옵션 ...)

옵션	기본값	기능
header	0	0번행을 열 이름 지정, None은 0번행 열 이름 지정 안함
Index_col	None	행 인덱스가 되는 열 번호 또는 열 이름 False : 인덱스 지정하지 않음 None : 0, 1, 2, ... 정수 인덱스
sep	,	텍스트 데이터 구분자 문자 지정
delimiterstr	None	텍스트 데이터 구분자 문자 지정
names		열 이름을 사용할 문자열 리스트
skiprows		처음 몇 줄을 skip할 것인지 설정
parse_dates	False	날짜/시간 데이터를 datetime64로 변환 여부 설정
Encoding		텍스트 인코딩 종류(UTF-X) 지정

PANDAS 데이터 파일 처리

◆ CSV 파일

```
import pandas as pd

# 2. CSV로 저장
csv_path = 'sample_data.csv'

# 3. read_csv() 시 parse_dates 적용
df1 = pd.read_csv( csv_path)
df2 = pd.read_csv( csv_path,
                   parse_dates=['date'])

# 4. 출력 확인
print("[df1]₩n", df1.dtypes)
print('------------')
print("[df2]₩n", df2.dtypes)
```

```
[df1]
date        object
product     object
price       int64
dtype: object

------------

[df2]
date        datetime64[ns]
product     object
price       int64
dtype: object
```

PANDAS 데이터 파일 처리

◆ EXCEL 파일

- 데이터를 표 형태로 정리, 계산, 분석, 시각화할 수 있는 파일
- 대표적인 확장자
  - .xlsx 일반적인 Excel 파일 형식
  - .xls 오랜된 Excel 파일 형식
  - .xlsm 매크로 기능 포함된 Excel 파일 형식

"ImportError: Install xlrd "

오류 발생시에는 xlrd 라이브러리 설치 필요

PANDAS 데이터 파일 처리

◆ EXCEL 파일

dataframe = Pandas.read_excel(“파일 경로(이름)”, 옵션 …)

옵션	기본값	기능
sheet_name	0	읽을 시트 지정. 0은 첫 번째 시트, "Sheet1"처럼 이름 지정 가능
header	0	몇 번째 행을 열 이름으로 사용할지 지정. None이면 열 이름 없이 읽음
index_col	None	특정 열을 행 인덱스로 사용할지 지정
usecols	None	읽을 열 범위 지정. 예: "A:D", "A,C,E"
names	None	직접 열 이름 지정
skiprows	None	처음 몇 줄 또는 특정 행을 건너뜀
nrows	None	읽을 행 개수 지정
dtype	None	열별 데이터 타입 지정
parse_dates	False	날짜/시간 데이터를 날짜형으로 변환
na_values	None	결측값으로 처리할 값 지정
engine	None	엑셀 파일을 읽는 엔진 지정. 보통 자동 선택

PANDAS 데이터 파일 처리

◆ JSON 파일

- JavaScript Object Notation 약자
- 자바스크립트에서 사용하는 객체 표기 방법
- 다양한 프로그래밍 언어에서 데이터 교환에 사용
- 인코딩/디코딩 표준으로도 사용

PANDAS 데이터 파일 처리

◆ JSON 파일

형태 : [{ 키:값, 키:값, 키:{ 키:값, 키:값 } }]

```
[ {  'id':1,
     'name':'jane',
     'data' : { 'major':'science', 'grade': 1 }
  }, { 'id':2,
     'name':'Tom',
     'data' : { 'major':'math', 'grade': 2 }
  } ]
```

PANDAS 데이터 파일 처리

◆ JSON 파일

dataframe = Pandas.read_json("파일 경로/이름.json", 옵션 ...)

옵션	기본값	기능
path_or_buf	필수	읽을 JSON 파일 경로 또는 JSON 문자열
orient	None	JSON 데이터 구조 지정
typ	'frame'	반환할 객체 유형 지정. 'frame' 또는 'series'
dtype	None	컬럼별 데이터 타입 지정
convert_dates	True	날짜 형식 데이터를 자동 변환할지 설정
encoding	'utf-8'	JSON 파일 인코딩 방식 지정
lines	False	JSON Lines 형식일 경우 True로 설정
nrows	None	lines=True일 때 읽을 줄 수 지정

PANDAS 데이터 파일 처리

◆ JSON 파일 처리

```
import pandas as pd

# JSON 파일 경로
json_file = 'data_lines.json'

# JSON 파일 읽기
data = pd.read_json(json_file, , lines=True)

# 데이터프레임 출력
print(data)
```

PANDAS PROGRAMMING

SERIES & DATAFRAME 구조 이해

구조화 데이터

◆ 시리즈(Series)

- 순차적으로 나열된 1차원 배열 형태
- 키(k)와 값(v)로 구성된 {K, V} 딕셔너리와 비슷

- 형식 : Index + Data

구조화 데이터

◆ 시리즈(Series)

KEY/INDEX		DATA
인덱스 라벨	Range Index	영화제목
2025	0	검은 수녀들
2019	1	극한 직업
2022	2	헤어질 결심
2020	3	남산의 부장들
2000	4	공동경비구역 JSA
2013	5	설국열차

구조화 데이터

◆ 시리즈(Series)

- **인스턴스/객체 생성**

```
pandas.Series( data=None,      → 데이터
               index=None,     → 데이터 식별 인덱스
               dtype=None,     → 데이터 타입/자료형
               name=None,      → 메타정보. 시리즈 이름
               copy=None )     → 독립 메모리 저장 여부
```

◆ 시리즈(Series)

▪ 속성들 – 기본 속성

분 류	속성명	설 명
라벨	index	• 시리즈의 인덱스(pd.Index) 객체
	name	• 시리즈 이름(메타데이터, DataFrame 변환 시 열 이름으로 사용)
	axes	• 축의 리스트([index] 반환)
	T	• 전치(Transpose). Series는 1차원이므로 원본과 동일
타입 및 메모리	dtype	• 데이터 타입(int64, float64, string, category, datetime64[ns] 등)
	array	• 내부 데이터를 담은 ExtensionArray 객체
	values	• 넘파이 배열 또는 ExtensionArray 뷰 (to_numpy() 권장)
	nbytes	• 데이터(값) 부분이 차지하는 메모리 바이트 수
	attrs	• 사용자가 임의로 부착 가능한 메타데이터 딕셔너리

◆ 시리즈(Series)

- **속성들 – 기본 속성**

분 류	속성명	설 명
형태 및 크기	shape	• (n,) 형태의 크기 튜플
	size	• 원소의 개수
	ndim	• 차원 수 (항상 1)
	empty	• 비어 있으면 True
인덱싱 및 접근자	loc	• 라벨 기반 인덱서
	iloc	• 위치 기반 인덱서
	at	• 단일 라벨 원소 접근(빠르고 엄격)
	iat	• 단일 위치 원소 접근(빠르고 엄격)

구조화 데이터

◆ 시리즈(Series)

딕셔너리 ➔ 시리즈

```
##- 모듈로딩
import pandas as pd

##- 데이터
##- Dict 타입의 경우 Key가 Index로 설정됨!
dict_data = {'a': 1, 'b': 2, 'c': 3}

##- Dict 이터 => 시리즈로 변환
sr = pd.Series(dict_data)

##- 시리즈 타입 출력
print(type(sr))

##- 시리즈 데이터 출력
print(sr)
```

```
a    1
b    2
c    3
dtype: int64
```

◆ 시리즈(Series)

▪ 속성 활용 / 값 출력

속성 읽기 => Series객체.속성명

속성 변경 => Series객체.속성명 = 새로운값

[예시]

```
sr = pd.Series([10, 20, 30])

print( "인덱스", sr.index )
print( "데이터", sr.values )
```

◆ 시리즈(Series)

▪ 원소/요소 선택

Series객체[정수인덱스] 또는 Series객체[인덱스라벨]

[예시]

```
sr = pd.Series([10, 20, 30])
print( "0번 원소 : ", sr[0]  )

sr = pd.Series([10, 20, 30], index=['n01', 'n02', 'n03'])
print( "n01 원소 : ", sr["n01"] )
```

◆ 시리즈(Series)

- **여러 개 원소/요소 선택**

[방법1] 리스트 기반

Series객체[[정수인덱스, 정수인덱스, ..., 정수인덱스]]

Series객체[[인덱스라벨, 인덱스라벡, ... , 인덱스라벨]]

[방법2] 슬라이싱 기반

Series객체[시작인덱스 : 끝인덱스: 간격]

구조화 데이터

◆ 시리즈(Series)

리스트 ➔ 시리즈

```
##- 데이터
list_data = ['2019-01-02', 3.14, 'ABC', 100, True]

##- 리스트 데이터 => 시리즈로 변환
sr = pd.Series(list_data)

##- 시리즈 타입 출력
print(f'sr 타입 : {type(sr)}')

##- 시리즈 속성 출력
print(f'sr 인스턴스===₩n{sr}')
print(f'series => {sr}')
print(f'index  => {sr.index}')
print(f'values => {sr.values}')
```

```
0    2019-01-02
1          3.14
2           ABC
3           100
4          True
dtype: object
```

구조화 데이터

◆ 시리즈(Series)

튜플 ➔ 시리즈

```
#- 데이터
tup_data = ('영인', '2010-05-01', '여', True)

##- 투플 => 시리즈로 변환(index 옵션에 인덱스 이름 지정)
sr = pd.Series(tup_data, index=['이름', '생년월일', '성별', '학생여부'])

##- 타입 출력
print(f'sr 타입 : {type(sr)}')

##- 속성 출력
print(f'sr 인스턴스===₩n{sr}')
print(f'series => {sr}')
print(f'index  => {sr.index}')
print(f'values => {sr.values}')
```

```
이름        영인
생년월일    2010-05-01
성별        여
학생여부    True
dtype: object
```

구조화 데이터

◆ 시리즈(Series)

튜플 ➔ 시리즈

```
## ---------------------------------------------
## 원소 선택
## ---------------------------------------------
# 원소를 1개 선택
print(sr[0])        ## sr의 1 번째 원소 선택 (정수형 위치 인덱스 활용)
print(sr['이름'])   ## '이름' 라벨 가진 원소 선택 (인덱스 이름 활용)
print('₩n')

# 여러 개의 원소 선택 (인덱스 리스트 활용)
print( sr[ [1, 2] ] )
print('₩n')
print( sr[ ['생년월일', '성별'] ] )
print('₩n')
```

구조화 데이터

◆ 시리즈(Series)

튜플 ➔ 시리즈

```
## ---------------------------------------------
## 여러 개의 원소를 선택 (인덱스 범위 지정)
## ---------------------------------------------
## 시작인덱스 : 끝인덱스, 끝인덱스 미포함
print( sr[ 1 : 2 ] )
print('₩n')

## 시작인덱스명: 끝인덱스명 , 모두포함
print( sr[  '생년월일' : '성별'  ] )
print( sr[  ['생년월일', '성별']  ] )
print('₩n')
```

구조화 데이터

◆ 데이터프레임(DataFrame)

- **행과 열(Series)**로 만들어진 **2차원 배열 구조**
- 통계 패키지 R의 데이터프레임에서 유래
- **여러 개의 시리즈(Series) 묶음**

- 형식 : Index + Columns + Datas

구조화 데이터

◆ 데이터프레임(DataFrame)

KEY/INDEX		COLUMNS		
인덱스 라벨	Range Index	학년	이름	학교
2025	0	1	홍길동	대구중
2019	1	3	베트맨	남산중
2022	2	2	임꺽정	남산중
2020	3	1	원더우먼	청도중
2000	4	1	마징가	대구중
2013	5	2	태권브이	앞산중

구조화 데이터

◆ 데이터프레임(DataFrame)

- **인스턴스/객체 생성**

```
pandas.DataFrame( data=None,        → 데이터
                  index=None,       → 데이터 식별 인덱스
                  columns=None,     → 컬럼 이름
                  dtype=None,       → 데이터 타입/자료형
                  copy=None )       → 독립 메모리 저장 여부
```

◆ 데이터프레임(DataFrame)

▪ 속성들 – 기본 속성

분 류	속성명	설 명
라벨	index	• 행 인덱스(pd.Index 객체)
	columns	• 열 인덱스(pd.Index 객체)
	axes	• [index, columns] 리스트 반환
	T	• 전치(Transpose). 행·열 교환된 DataFrame 반환
타입 및 메모리	dtypes	• 각 열의 데이터 타입(dtype) 시리즈
	array	• 내부 데이터를 담은 ExtensionArray 뷰 (Series 단위와 유사)
	values	• DataFrame 전체를 넘파이 배열 또는 ExtensionArray 형태로 반환
	nbytes	• 데이터(값) 부분이 차지하는 메모리 바이트 수
	attrs	• 사용자가 임의로 부착 가능한 메타데이터 딕셔너리

◆ 데이터프레임(DataFrame)

▪ 속성들 – 기본 속성

분 류	속성명	설 명
형태 및 크기	shape	• (행, 열) 형태의 크기 튜플
	size	• 전체 원소 수 (행 × 열)
	ndim	• 차원 수 (항상 2)
	empty	• 비어 있으면 True
인덱싱 및 접근자	loc	• 라벨 기반 인덱서 — 행·열 이름으로 접근
	iloc	• 위치 기반 인덱서 — 행·열 번호로 접근
	at	• 단일 라벨 원소 접근(빠르고 엄격)
	iat	• 단일 위치 원소 접근(빠르고 엄격)

구조화 데이터

◆ 데이터프레임(DataFrame)

딕셔너리 ➔ 데이터프레임

```
##- 모듈로딩
import pandas as pd

##- 열이름 key, 리스트 value로 갖는 딕셔너리 데이터
##- 2차원 배열
dict_data = {'c0':[1,2,3],  'c1':[4,5,6], 'c2':[7,8,9], 'c3':[10,11,12], 'c4':[13,14,15]}

##- 딕셔너리 => 데이터프레임으로 변환
df = pd.DataFrame(dict_data)

##- 데이터프레임 속성 출력
print(f'df 인스턴트 ===₩n{df}')
print(f'df 타   입 : {type(df)}')
```

	c0	c1	c2	c3	c4
0	1	4	7	10	13
1	2	5	8	11	14
2	3	6	9	12	15

컬럼

인덱스

◆ 데이터프레임(DataFrame)

- **속성 활용 / 값 출력**

속성 읽기 => DataFrame객체.속성명

속성 변경 => DataFrame객체.속성명 = 새로운값

구조화 데이터

◆ 데이터프레임(DataFrame)

2차원 리스트 ➔ 데이터프레임

```
##- 모듈로딩
import pandas as pd

##- 2차원 리스트 => 데이터프레임으로 변환
df = pd.DataFrame( [ [15, '남', '덕영중'],
                     [17, '여', '수리중'] ],
                     index=['준서', '예은'],
                     columns=['나이', '성별', '학교'] )

##- 데이터프레임 속성 출력
print(f'df 인스턴스===₩n{df}')
print(f'df 타   입 : {type(df)}')
```

	나이	성별	학교
준서	15	남	덕영중
예은	17	여	수리중

구조화 데이터

◆ 데이터프레임(DataFrame)

2차원 리스트 ➔ 데이터프레임

```
# 행 인덱스, 열 이름 변경하기
df.index=['학생1', '학생2']
df.columns=['연령', '남녀', '소속']

print(df)              #데이터프레임
print(df.index)        #행 인덱스
print(df.columns)      #열 이름
```